HW2: Traditional Software Design vs Function-as-a-Service

Objective

This homework aims to deepen your understanding of modern software architecture choices and their implications on maintainability, security, extensibility, and performance.

You will design and implement a small system using two different architectural approaches:

1. A traditional application design

2. A Function-as-a-Service (FaaS) approach

Through this assignment you will analyze how architectural decisions affect the effort required to implement features, modify the system, maintain security, and evaluate performance.

The goal is not only to write working code, but also to critically evaluate architectural tradeoffs.

Assignment Instructions

You must first select a realistic system scenario.

Examples include (but are not limited to):

- Hospital management system
- Hotel management system
- Airport operations system
- University course management

Your system must contain multiple interacting functionalities that simulate a realistic operational environment.

Example functionalities in a hospital system could include:

- Scheduling operating rooms
- Managing doctor shifts
- Assigning nurses to hospital departments
- Handling emergency room capacity
- Tracking patient admission and discharge
- Allocating medical equipment

The system should contain at least 7 meaningful operations or services.

Part 1 - Traditional System Implementation

Implement your system using a traditional architecture.

This may include:

- A monolithic program
- A modular service-based design
- A traditional server application

Requirements:

- Implement the core system logic.
- Organize the code using standard programming structures such as classes, modules, and functions.
- Your implementation should allow the system to perform the functionalities you defined.

Focus on writing code that represents how such a system would normally be implemented in a traditional software architecture.

Part 2 - Function-as-a-Service (FaaS) Implementation

Re-implement the same system using a Function-as-a-Service style design.

Instead of a single program, the system should be composed of independent functions that perform specific tasks.

Examples:

- schedule_operation_room()
- assign_nurse_to_department()
- update_doctor_shift()
- handle_patient_admission()

Each function should represent an independent service trigger that can be executed separately.

Your implementation does not need to run on a real cloud provider, but the structure should clearly follow FaaS principles, such as:

- Stateless execution
- Isolated functionality
- Minimal dependencies between functions

Part 3 - Feature Extension Challenge

After implementing both architectures:

You may think of a new feature yourself or use an AI assistant to propose a new complex feature for your system.

Examples for a hospital system:

- Automatic reallocation of staff during emergencies

- Predictive scheduling based on historical patient load
- Dynamic operating room prioritization

Your task is to evaluate:

- How difficult it would be to add this feature in the traditional architecture
- How difficult it would be in the FaaS architecture

You should discuss:

- How many parts of the system must change
- How risky the modification is
- Which architecture is easier to extend

You do not need to fully implement the feature, but you should show a partial implementation or design changes demonstrating the difference.

Part 4 - Performance Evaluation

Using perf (or another profiling tool), evaluate the runtime characteristics of both implementations.

Examples of metrics you may examine:

- Execution time
- CPU cycles
- Context switches
- Memory usage
- System calls

Run a workload that exercises multiple system operations and compare the results between the two architectures.

Explain:

- Which architecture performed better
- Why the observed behavior makes sense

~~Create a FlameGraph for each architecture and write your conclusions.~~

~~Part 5 - Security and Maintainability Discussion~~

~~Analyze the differences between the two approaches in terms of (pick one more topic in addition to security) :~~

- ~~Security~~

    ~~Examples to discuss:~~

    - ~~Attack surface~~
    - ~~Isolation between components~~
    - ~~Risk of bugs affecting the entire system~~
- ~~Maintainability~~
- ~~Code complexity~~
- ~~Ease of debugging~~

~~You should support your arguments with concrete observations from your implementations.~~

Submission Requirements

Submit a single file named:

HW2.zip

The ZIP file must contain the following structure:

HW2.zip

-------report.pdf # maximum 6 pages containing the written analysis.

-------ids.pdf # contains the names and IDs of the students

-------script.sh

-------Traditional/

-------FaaS/

-------optional files (if needed):

<u>script.sh</u>

A shell script containing the commands used for:

- Running the system
- Executing test scenarios
- Running performance profiling

<u>Traditional/</u>

Source code implementing the traditional architecture.

<u>FaaS/</u>

Source code implementing the Function-as-a-Service architecture.

Languages allowed:

- C
- C++
- Python

Grading Criteria

Correctness and completeness - 70%

- Both implementations work
- The architectures are clearly different
- Profiling and analysis are performed correctly

Architectural insight and originality — 30%

- Complexity of the chosen system
- Depth of the architectural comparison
- Quality of the reasoning and analysis

Rules

You must not:

- Copy implementations from online repositories
- Use prebuilt frameworks that hide the architectural differences
- Submit the same system design as another team

You may use AI tools for architectural discussion, but you must clearly describe how you used them.

Recommendations

Choose interesting real-world systems.

Think about how software evolves over time.

Reflect on why different architectures exist.